

SIMATS ENGINEERING
Department of Computer Science and Engineering
ASSESSMENT TOOL 1-ANALYTICAL PROBLEM SOLVING

Course Code:

CSA12

CO Assessed:

Weightage:

CO2 - Precision (BL3)

45%

CO2 Statement:

Student Name:

Section / Batch:

Course Title:

Assessment:

Total Marks:

Computer Architecture Analytical Problem Solving

100

Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/*non*- restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.

Sheen Ivin Clement Reg. No.:

Instructions to Students

2023 2027

Date:

192371023 03-08-2026

Identify the appropriate arithmetic algorithm or number representation required for **the** given computational problem.

Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm. **Apply** Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate **results**. **Compute binary** multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms. Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A computer system uses 8-bit 2's complement representation to process signed integer arithmetic. Consider two numbers: -52 and +19. Analyze in detail how the processor performs both addition and subtraction, including binary conversion, sign extension, intermediate steps, and the final result. Determine whether **overflow** occurs in each operation and explain how the processor detects and handles overflow.
2. A **modern** processor is designed with an 8-bit carry look-ahead adder to accelerate arithmetic operations. **Given two binary** operands, analyze how the generate and propagate signals are produced, how carry bits are **calculated** simultaneously, and compare the execution time of both adders. Justify why CLA is preferred in high-performance processors.
3. A digital **processor** performs signed multiplication using Booth's algorithm for efficient arithmetic computation. Multiply +11 and -7, and analyze each iteration of the algorithm by showing the contents of **the** accumulator, multiplier register, Booth bit, and the arithmetic right shifts. Explain how Booth's algorithm minimizes **the** number of addition and subtraction operations while correctly handling signed operands.
4. A binary **arithmetic** unit supports both restoring and non-restoring division techniques for integer division. Analyze the execution of the division of 25 ÷ 4 using both methods by showing the intermediate remainder values, **quotient** generation, and register contents after each step. Compare the algorithms based on **execution time**, hardware complexity, and computational efficiency, and explain why one method is **preferred over the other**.
5. A floating-point **unit** (FPU) processes real numbers according to the IEEE 754 standard. Analyze the representation of the decimal number +26.75 in both single-precision (32-bit) and double-precision (64-bit) formats by identifying the sign, exponent, and mantissa fields. Further, analyze the floating-point addition of 26.75 and -8.5, explaining exponent alignment, mantissa operations, normalization, rounding, and the **effect** of precision on **the** final result.

Code of Conduct

1 Shen Ivin Clement / 19234/123 (Name / Reg No) certify *that this submission is my original work and* that I have adhered to the guidelines specified for this assessment. I understand that any violation of **academic integrity** rules will result in disciplinary action.

Signature of the Student:

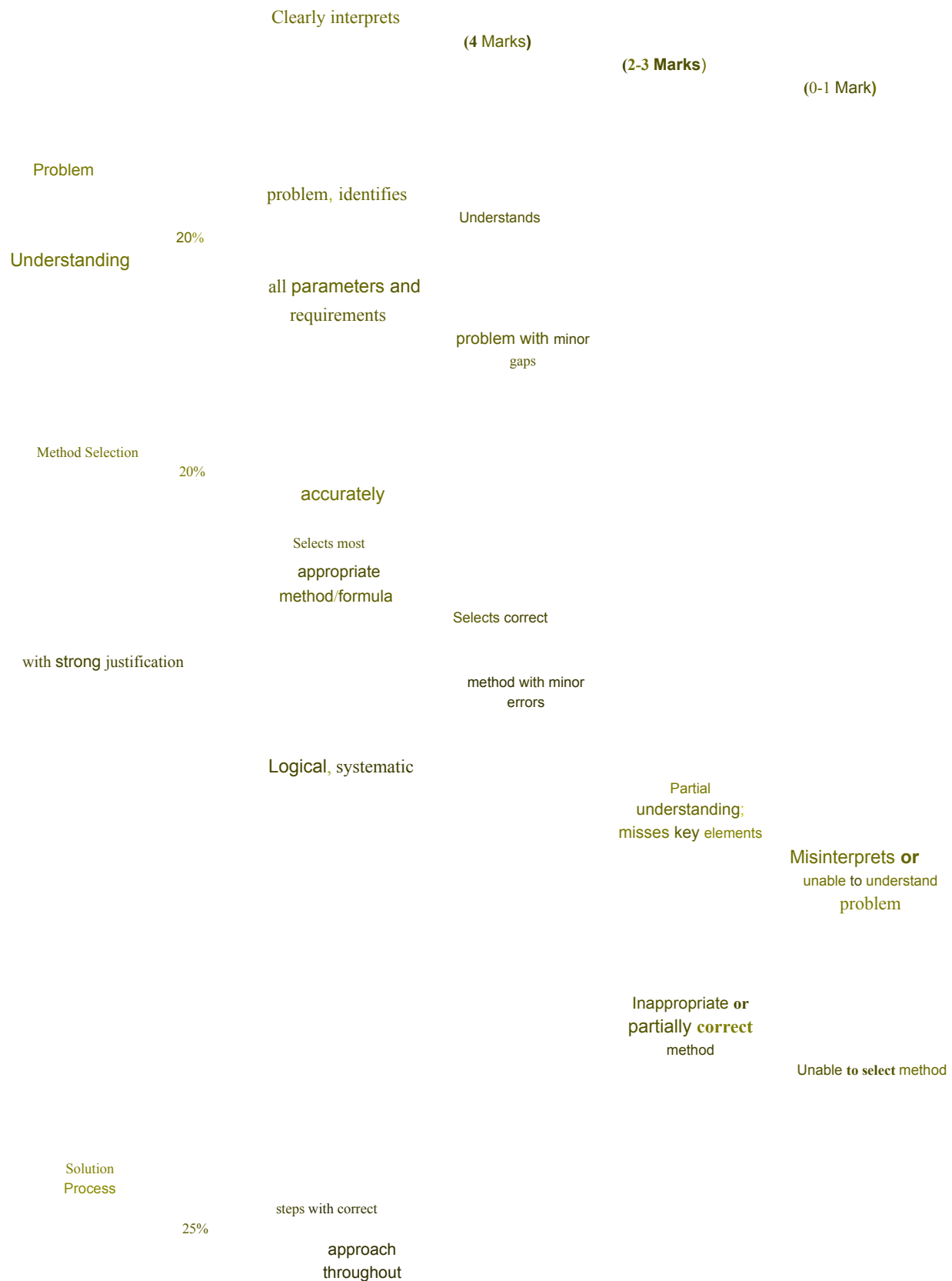
ebos

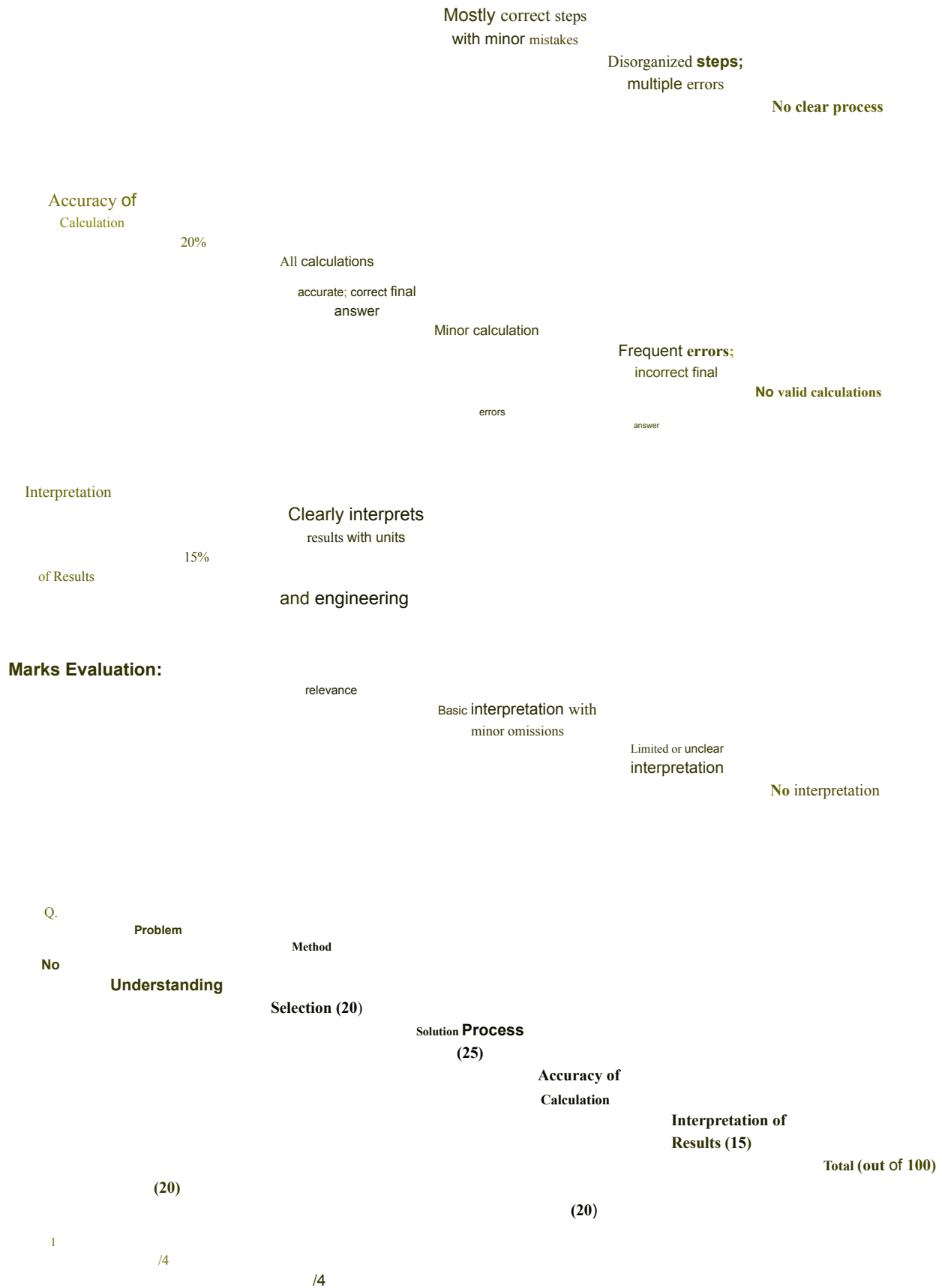
RUBRICS FOR CALCULATION

Criteria

Weightage

Level 4-Exemplary Level 3 - Proficient Level 2 - Developing Level
1 - Beginning
(5 Marks)





Bimory

:=

8-bit is complement -52

$$+52 = 0011\ 0100$$

I 's

Complement

$$\begin{array}{r} -52 \\ = 11001\ 100 \end{array}$$

$$+14=00010011$$

Addition

$$\begin{array}{r} + \\ 11\ 001100\ 00010011 \\ 11011111 \end{array}$$

Subtraction:-

$$\begin{array}{r} -52+19: \\ = -33 \end{array}$$

$$\begin{array}{r} - \\ -52 \end{array}$$

$$119 = -52 + (-19)$$

$$+19=$$

$$-19=$$

$$11001100\ 11101101$$

$$110\ 111001$$

Discard Covey:

$$10111\ 004$$

Omaflow =>

No

$$00010011$$

$$\underline{1}\ 1\ 1\ 0\ 1\ 1\ 0\ 1$$

2) 8 bit lorey took - Ahead Adder

For each bit:-

Carry!"

G1 = A; Bi

P1 = A; & B1

Pi

-71

Cit1 = G; +
PiCi

Clambake louiss

for each lovy as im

RCA

Carey
Scquential

Ваша

Simple
Hardware

Eg:- C1 = G₀ + P₀
Co

C2

-1

Од трастот раРосо

in pacalled instead of
wanting

"Pipple Corey Addo.

CLA

love
lakemlakd

quickly

Faster

More Hardware

3

Lower

cost

Higher Performance

-Performance

Powers

CLA is preferred in high

-power

because it significantly reduces logic
propagation
delay

delay

Booth Multiplication

Multiplication $+11 \times -7$

$+11 = 01011$

$-7 = 11001$

Initial:

$A = 00000$

$Q = 11001$

$1-1-$

$M =$

01011

Booth Aules : –

12o. Q-1

00
01
10
11

Operation

No operation

A = A+M

A = A-M

Alo Operation

4

Division 05 ÷ 4

255 11001,

Expected Result: Q

= 6

R = 1

Restoring

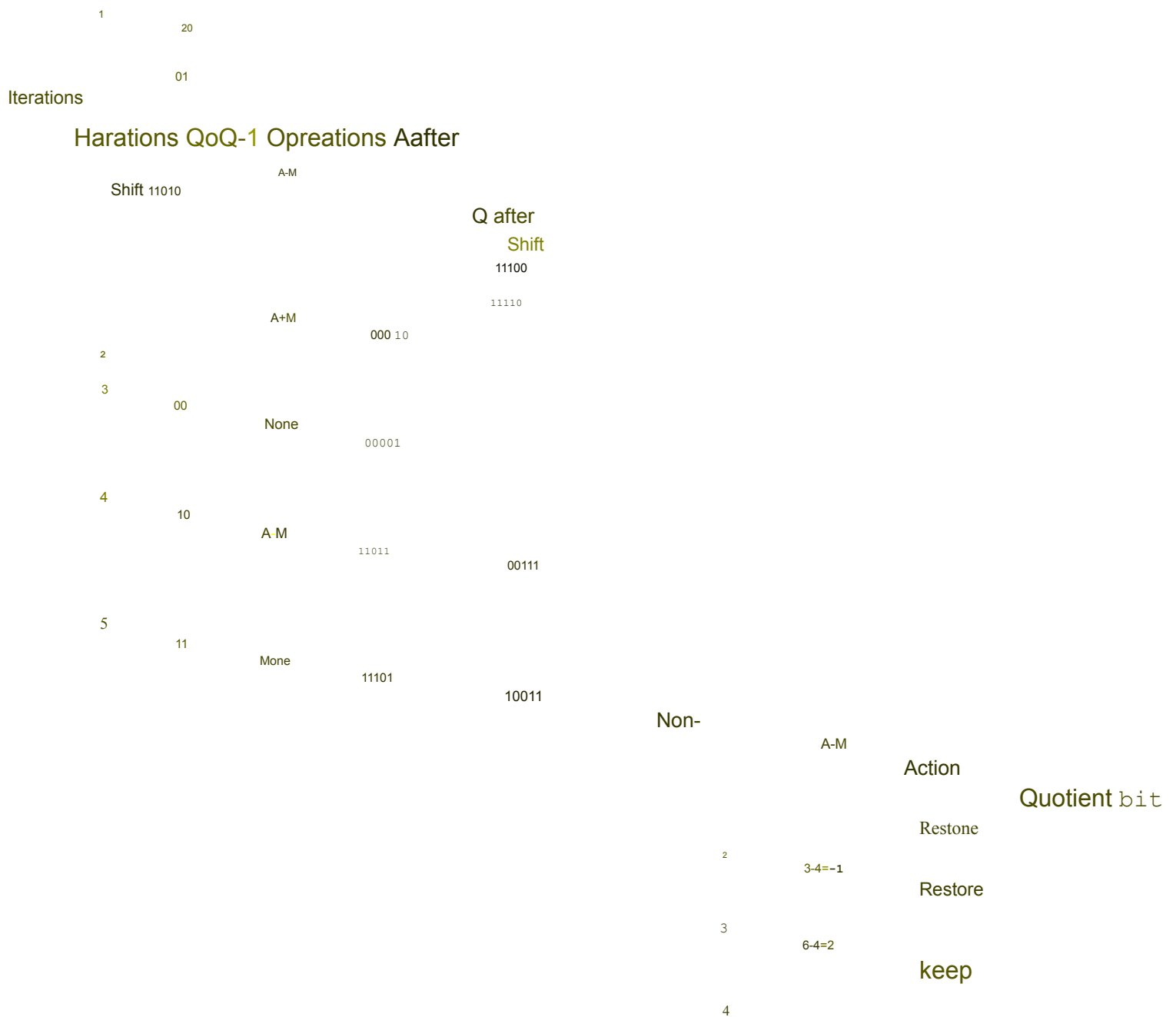
Dinicion :-

Iteration

1

1-4=-3

4=00100



$$4-4=0$$

keep

1

815

$$1-4=-3$$

Restore

01111

Q

$$00110=6 \quad R = 0000 \quad 01 = 1$$

Restoring Division

Instead of immediately restoring

negative Remainder :-

Positive remainder Subtraction

division

Negative Remainder & add divisor

A Q

111011 0011

Advantage :-

$$11 \times (-7) = -77$$

Booth techies additions / Subtractions by efficiently handling sign

It's and directly Support signed 2's

Complement

number.

Abta

B

iterations:-

$$Q = 6$$

$$R=1$$

Comparison

Restoring

Restoring negations
remainder
immediately

Non-Restoring

Arithmetic immediate
restoration

Arithmetic Operation

Double Precision:-
64 bit

$$= 4 + 1023 = 1027 = 1000\ 000\ 001$$

Sign = 0

Exponent

Fraction

101 011 followed by row

More Operations

Lowes

Generally

faster

Simple
Control

More
efficient

Normalize:-

Non-
Gestorming

division browse it reduces

Almuniary autoration
quations.

IEEE 754: + 26.75

26.75=1101 0.11

2

0x 403 Ac 000 000 000 00

Floating-Point
Addition

26.75+ (-8.5)

26.75= 1- 101011 x 24

-8.5 =

Align exponents

Subtract :-

-1.0001823

1. 101011 X24 -0.100010 x 27

1. 101011

-0. 100040

Normalize:

1.101 011

x24

32

bit

1. 001001

Thus: -

1. 001001 $\times 2^4$

18-25

Single Precision

Sign Exponent

Fraction

$$= 4 + 127 = 131$$

10000011

- 101 011 000 000 000 00000

... 0 | 100000 11 | 101 0 1 1
000 000 000 000000

$$-52 + 19 = -33$$

-52-

19= - 71

11x (-7)

- 77

$$25=4=6Rq$$

$$26.75 + (-8 \cdot 5) =$$

18. 25